

Part. 1

IPC

Inter-Process Communication

프로세스 간 통신

발표자 Janghwan Kim (Harley)

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증



- 01 전역변수 + 뮤텝스 / 세마포어
- 02 공유메모리 + 링버퍼 (메시지 큐)
- 03 TCP / IP 소켓

발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제인 Inter-Process Communication, 프로세스 간 통신의 약자인 IPC 를 이용해 멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현에 관한 내용을 발표하겠습니다.

목표

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증

01

쓰레드 / 프로세스 개념 및 통신 방법

개념

리눅스 환경에서 프로그램 · 프로세스 · 쓰레드가 무엇이고, 서로 어떻게 데이터를 주고받는지 정리한다.

02

IPC 개념 및 뮤텍스 / 세마포어

개념

UDP · TCP/IP · MessageQ · 링버퍼 · 공유메모리의 특징과 사용 상황, 장 / 단점. 그리고 이를 안전하게 묶어주는 동기화 수단.

03

데이터 송 / 수신 기능 구현 및 코드 정리

구현

송신 · 수신 쓰레드를 구성해 1 부터 100 까지 0.1 초 간격으로 주고받는다.
ToDo#1 쓰레드 간 · ToDo#2 프로세스 간 · ToDo#3 TCP / IP

오늘 발표는 1 → 2 → 3 순서로, 개념을 먼저 정리하고 그 개념이 코드 어디에 쓰였는지 이어서 보여드립니다.

발표 대본

목표 · 약 25초

오늘 다룰 내용은 크게 세 가지입니다.

첫 번째로, 리눅스 환경에서의 쓰레드 / 프로세스 개념과 통신 방법을 설명하고,

두 번째로 IPC 개념과 동기화 기법인 뮤텍스, 세마포어를 설명하겠습니다.

세 번째로는, 해당 개념들을 이용해 데이터 송수신 기능을 구현했고, 실행 결과까지 보여드리겠습니다.

01

쓰레드 / 프로세스 개념 및 통신 방법

리눅스 환경 기준

1-1

프로그램 · 프로세스 · 쓰레드

무엇이 다르고, 왜 그 차이 때문에 통신 방법이 갈리는가

1-2

통신 방법

쓰레드 간 · 프로세스 간에 쓸 수 있는 수단과 선택 기준

발표 대본

섹션 1 · 약 20초

첫 번째 주제입니다. 리눅스 환경 기준의 쓰레드와 프로세스 개념, 그리고 통신 방법을 설명하겠습니다.

우선 프로그램 / 프로세스 / 쓰레드가 무엇이 다른지 정리하고, 그 차이에 따라 쓸 수 있는 통신 방법이 어떻게 갈리는지 설명하겠습니다.

1-1 프로그램 · 프로세스 · 쓰레드

무엇을 공유하고 무엇을 따로 갖는지가 통신 방법을 결정한다

1 프로그램 Program

디스크에 저장된 실행 파일. 아직 실행되지 않은 명령어와 데이터의 묶음이다.

□ 정적 — 메모리도 자원도 할당되지 않은 상태

2 프로세스 Process

실행 중인 프로그램. 커널이 독립된 가상 주소공간과 자원(PID · Process ID, 파일 디스크립터)을 붙여준 단위다.

□ 서로의 메모리를 볼 수 없다 → 그래서 IPC 가 필요하다

3 쓰레드 Thread

프로세스 안의 실행 흐름. Code · Data · Heap 은 공유하고 Stack 과 레지스터(PC : Program Counter · SP : Stack Pointer)만 따로 갖는다.

□ 전역변수로 바로 주고받는다 → 대신 동기화가 필요하다

리눅스에서는 프로세스도 쓰레드도 커널 입장에선 task_struct 하나. clone() 에 무엇을 공유할지만 다르게 준다.

프로세스 A (독립된 가상 주소공간)

쓰레드끼리 공유하는 영역

Code

Data (전역변수)

Heap

파일 디스크립터

쓰레드 1 (송신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

쓰레드 2 (수신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

전역변수를 통해 복사 없이 바로 전달 · 단, 동시 접근은 뮤텍스로 보호



IPC 필요

프로세스 B

주소공간이 다르다 → 직접 접근 불가

발표 대본

1-1 프로그램 · 프로세스 · 쓰레드 · 약 1분 5초

프로세스와 쓰레드의 차이를 설명하기에 앞서 프로그램이라는 것을 먼저 알아야 하는데, 프로그램은 디스크에 저장된 실행 파일입니다. 아직 실행되지 않은 명령어와 데이터의 묶음이고, 메모리도 자원도 할당되지 않은 정적인 상태입니다.

프로세스는 그 프로그램이 실행된 것입니다. 커널이 독립된 가상 주소공간과 자원들을 붙여 준 단위입니다. 프로세스끼리는 서로의 메모리를 볼 수 없다는 특징을 가지고 있습니다. 바로 이 점 때문에 IPC 라는 별도의 수단이 필요합니다.

쓰레드는 프로세스 안의 실행 흐름입니다. 우측 그림과 같이 Code / Data / Heap / 파일 디스크립터는 쓰레드끼리 공유하고, 스택과 레지스터만 쓰레드마다 따로 갖습니다. 그래서 쓰레드끼리는 전역변수 하나로 바로 데이터를 공유할 수 있습니다. 대신 두 쓰레드가 같은 변수를 동시에 건드리면 값이 깨지기 때문에 동기화가 필요합니다.

1-2

통신 방법

주소공간을 공유하느냐, 그리고 얼마나 멀리 보내느냐가 쓸 수 있는 수단을 정한다



고르는 기준은 하나 — 얼마나 멀리 보내야 하는가

가까울수록 빠르다. 그만큼 커널이 해 주던 경계와 순서, 동기화가 내 코드로 넘어온다.

본 과제는 이 세 칸을 그대로 **ToDo#1 · ToDo#2 · ToDo#3** 으로 구현했다

발표 대본

1-2 통신 방법 · 약 1분 10초

통신 방법은 결국 얼마나 멀리 보내야 하는가 하나로 갈립니다.

첫째, 같은 프로세스 안이면 주소공간을 공유하기 때문에 전역변수나 공유 자료구조를 그대로 씁니다. 메모리 주소만 넘기면 되니 가장 빠릅니다. 데이터가 깨지거나 데드락이 안 나려면 동기화가 필요합니다. 그리고 한 쓰레드가 죽으면 프로세스 전체가 같이 죽습니다. 이 내용이 오늘의 ToDo#1 입니다.

둘째, 같은 PC 내에 다른 프로세스이면 주소공간이 분리돼 있어 반드시 커널이 만든 통로를 거칩니다. 메시지 큐는 커널이 메시지 경계와 순서를 지켜 주고, 공유메모리는 같은 물리 메모리를 양쪽에 붙여 놔서 옮겨 담을 일이 없습니다. 이 내용이 오늘의 ToDo#2 입니다.

셋째, 다른 PC 나 다른 프로그램이면 소켓이 필요합니다. 이 중 유일하게 장비를 넘어가고, 대신 연결과 유실, 순서를 직접 처리해야 해서 가장 느립니다. 이 내용이 오늘의 ToDo#3 입니다.

02

IPC 개념 및 뮤텍스 / 세마포어

각 수단의 특징 · 어떤 상황에서 쓰는지 · 장 / 단점

2-1

UDP · TCP / IP

네트워크를 건너는 통신. 신뢰성이나 속도나

2-2

MessageQ · 링버퍼 · 공유메모리

같은 PC 안에서 데이터를 옮기는 세 가지 도구

2-3

뮤텍스 · 세마포어

공유 자원을 안전하게 만들어 주는 동기화 수단

발표 대본

섹션 2 · 약 25초

두 번째 주제입니다. IPC 수단들의 개념과 동기화 기법인 뮤텍스, 세마포어를 설명하겠습니다.

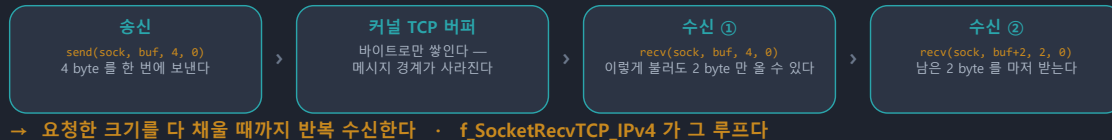
세 부분으로 나눠서, 먼저 UDP 와 TCP/IP 를 비교하고, 같은 PC 안에서 데이터를 옮기는 메시지 큐 · 링버퍼 · 공유메모리를, 마지막으로 이들을 안전하게 동기화해 주는 뮤텍스 · 세마포어 기법을 보겠습니다.

2-1 UDP · TCP / IP

둘 다 IP 위에 올라간다. 차이는 하나 — 신뢰성을 커널이 책임지느냐, 앱이 책임지느냐

구분	TCP / IP Transmission Control Protocol / Internet Protocol	UDP User Datagram Protocol
연결	3-way handshake 로 맺고 시작한다	없음 — 주소만 알면 바로 보낸다
신뢰성	커널이 보장 (재전송 · 순서 · 중복)	보장 없음 — 앱이 직접 처리
데이터 단위	바이트 스트림 — 경계가 없다	데이터그램 — 보낸 크기 그대로
통신 상대	1 : 1 만 가능	1 : N 브로드캐스트 · 멀티캐스트
헤더 · 지연	20 byte · 재전송으로 지연이 된다	8 byte · 가볍고 지연이 낮다
쓰는 곳	파일 전송 · 명령 / 제어 · 로그 수집	센서 · 음성 / 영상 스트리밍
과제 적용	ToDo#3 의 데이터 송 / 수신	TCP Sync 라인체크에만 사용

send 한 번이 recv 한 번으로 오지 않는다



→ 요청한 크기를 다 채울 때까지 반복 수신한다 · f_SocketRecvTCP_IPv4 가 그 루프다

발표 대본

2-1 UDP · TCP / IP · 약 1분 30초

UDP 와 TCP/IP 둘 다 IP 를 통해서 통신하며, 신뢰성을 커널이 책임지느냐 앱이 책임지느냐로 나뉩니다.

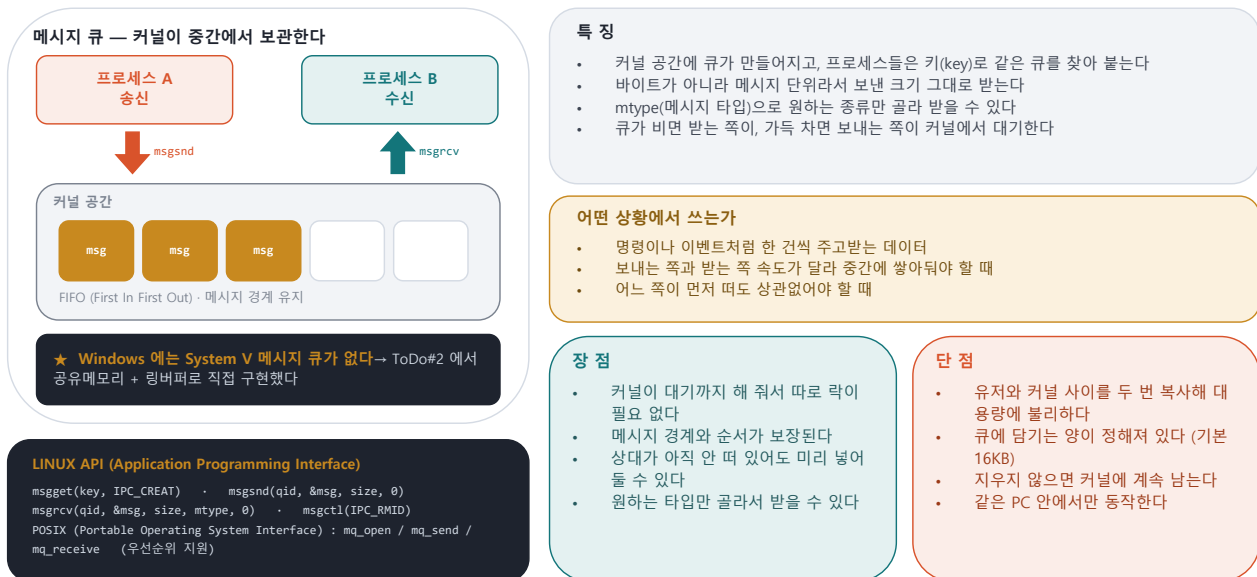
TCP 는 3-way handshake 로 연결을 맺고 시작하며, 커널이 신뢰성을 보장한다는 특징이 있습니다. 그리고 데이터를 바이트 스트림 단위로 보내기 때문에 경계가 없습니다.

반면 UDP 는 연결 절차 없이 주소만 알면 바로 보냅니다. 그렇기 때문에 신뢰성이 보장되지 않는 특징이 있습니다. 그리고 데이터를 데이터그램 단위로 보내기 때문에 보낸 크기 그대로 전달된다는 특징이 있습니다.

아래의 검은색 박스에서 설명하는 것처럼 오늘 ToDo 세 번째 주제에서 TCP/IP 를 사용하게 되는데, send 함수로 4byte 를 한 번에 보낸다고 해도 커널 TCP 버퍼에서는 바이트 단위로 쪼개게 되어, 수신하는 입장에서 4byte 로 recv 한다고 해도 2byte 만 올 수가 있습니다. 그래서 TCP/IP 에서는 요청한 크기를 다 채울 때까지 반복 수신하는 기능이 있습니다. 이 기능이 과제 내용과 같이 첨부 파일로 받았던 SocketUtility 소스 파일 내에 f_SocketRecvTCP_IPv4 함수 내에 그대로 구현되어 있습니다.

2-2 메시지 큐 Message Queue

커널이 메시지 단위로 보관해 주는 큐. 보내는 쪽과 받는 쪽이 동시에 떠 있지 않아도 된다



발표 대본

2-2 ① 메시지 큐 · 약 1분 50초

메시지 큐는 커널이 관리하는 메시지 단위의 큐입니다.

그림처럼 큐 자체가 커널 공간에 만들어지고, 프로세스들은 키를 가지고 동일한 큐를 찾아 붙습니다. 프로세스 A가 msgsnd로 넣고, 프로세스 B가 msgrcv로 FIFO 선입선출 원리로 꺼냅니다. 그리고 메시지 큐는 바이트가 아니라 메시지 단위라서 보낸 크기 그대로 받습니다. 또한 수신측에서 msgrcv 함수로 값을 읽어올 때 mtype 매개변수를 설정함으로써 원하는 종류의 데이터만 골라 받을 수 있습니다. 그리고 큐가 비어 있으면 받는 쪽이 커널에서 대기하고, 가득 차 있으면 보내는 쪽이 커널에서 대기하게 됩니다. 이 말은 즉, 메시지 큐에는 커널에서 자동으로 동기화를 해 주는 기능이 들어 있다는 뜻입니다.

그래서 커맨드나 이벤트처럼 한 건씩 데이터를 주고받을 때 메시지 큐를 사용하고, 보내는 쪽보다 받는 쪽이 처리 속도가 느려서 중간에 데이터를 쌓아둬야 할 때, 그리고 송신과 수신 어느 쪽이 먼저 동작하더라도 상관없어야 할 때 사용됩니다.

단점은 보내는 쪽이 큐에 복사해서 저장하고 그 데이터를 다시 받는 쪽에 복사해서 총 두 번 복사하기 때문에 대용량에 불리합니다. 그리고 큐에 담기는 양이 정해져 있다는 단점이 있습니다.

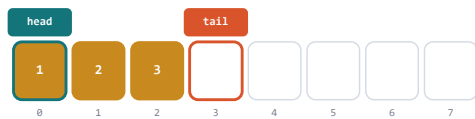
메시지 큐가 ToDo 두 번째 주제인데, Linux에는 System V라는 메시지 큐 기능이 있는데 Windows에는 메시지 큐 기능이 없어서 공유메모리와 링버퍼로 메시지 큐 기능을 직접 구현했습니다.

2-2

링버퍼 Ring Buffer

고정 크기 배열을 원형으로 돌려 쓰는 자료구조. IPC 수단이 아니라 데이터를 쌓아 두는 방법이다

고정 크기 배열 8 칸을 원형처럼 돌려 쓴다



끝에 닿으면 $(tail + 1) \% N$ 으로 0 번 슬롯으로 되돌아간다

head — 읽는 위치
수신 쪽이 여기서 꺼낸다

tail — 쓰는 위치
송신 쪽이 여기에 넣는다

```
ring[tail] = msg; tail = (tail+1) % N; count++;
msg = ring[head]; head = (head+1) % N; count--;
```

본 과제 적용

쓰레드 간 8 슬롯 전역 링버퍼 (lpcThread.c)
프로세스 간 64 슬롯 링버퍼를 공유메모리 위에 (lpcMsgQ.c)

특징

- 고정 크기 배열 + 읽는 위치(head) / 쓰는 위치(tail) 두 개의 인덱스
- 끝에 닿으면 나머지 연산으로 처음으로 되돌아간다 → $tail = (tail + 1) \% N$
- 개수가 늘어도 넣고 빼는 시간이 그대로다 ($O(1)$), 실행 중 동적 할당도 없다
- 비었는지 찼는지 구별하려면 개수를 세거나 한 칸을 비워 둔다

어떤 상황에서 쓰는가

- 넣는 쪽과 꺼내는 쪽 속도가 다를 때 사이에 두고
- 시리얼 · 네트워크 수신 버퍼, 오디오 · 센서 스트림, 로그 버퍼
- 실행 중에 메모리를 할당하면 안 되는 실시간 코드

장점

- 넣기 / 빼기 시간이 개수와 무관하게 일정 ($O(1)$)
- 메모리 고정 — 단편화도 할당 지연도 없다
- 연속 메모리라 캐시에 유리하다
- 공유메모리 위에 그대로 올릴 수 있다

단점

- 크기가 고정이라 가득 차면 기다리거나 버려야 한다
- head · tail 갱신 자체가 임계구역이다
- 동기화를 직접 붙여야 한다
- 가변 길이 데이터는 담기 번거롭다

발표 대본

2-2 ② 링버퍼 · 약 1분 30초

그래서 메시지 큐를 윈도우에서 구현하기 위한 링버퍼와 공유메모리를 설명드릴 건데, 먼저 링버퍼는 앞에 설명한 IPC 수단과는 달리 단지 데이터를 담은 자료구조입니다.

지금 그림이 원형이 아닌 일자로 되어 있긴 한데, 원형으로 동글게 둘러앉아 있는 모습이라고 생각해 주시면 되겠습니다. 고정 크기 배열에 읽는 위치 head 와 쓰는 위치 tail 두 개의 인덱스로 데이터에 접근이 가능합니다. 데이터가 점점 쌓이다가 마지막 인덱스에 닿게 되면 0번 인덱스 처음으로 돌아갑니다. 개수가 늘어도 head 나 tail 인덱스로 데이터에 한 번 접근하면 되기 때문에 $O(1)$ 이라는 시간 복잡도를 가집니다.

링버퍼는 앞 장에서 설명한 메시지 큐 내용과 동일하게 넣는 쪽과 꺼내는 쪽의 처리 속도가 다를 때 사용하고, 시리얼 통신이나 센서 데이터 스트리밍 등을 위해 사용하며, 실행 중에 메모리를 할당하면 안 되는 매우 빨리 동작해야 하는 실시간 코드일 때 사용합니다.

단점은 크기가 고정이라 넣는 쪽이 꺼내는 쪽의 인덱스를 추월했을 때 오래된 데이터를 덮어쓰거나 꺼내는 쪽을 기다려야 한다는 단점이 있고, head 와 tail 갱신 자체가 임계 구역이라 동기화를 직접 해야 합니다.

2-2 공유메모리 Shared Memory

두 프로세스가 각자의 주소공간에 같은 물리 메모리를 붙인다. 붙고 나면 그냥 포인터로 읽고 쓴다



발표 대본

2-2 ③ 공유메모리 · 약 1분

공유메모리는 두 프로세스가 각자의 주소공간에 같은 물리 메모리를 붙여 놓고 쓰는 방식입니다.

그림처럼 프로세스 A 와 B 가 각각 다른 가상 주소로 접근하지만, 실제로는 같은 물리 페이지를 보고 있습니다. 커널은 붙여 주는 일까지만 하고, 그 뒤 읽고 쓸 때는 시스템 콜이 전혀 개입하지 않습니다. 데이터를 옮겨 담지 않으니 IPC 중에 가장 빠릅니다.

그래서 영상 프레임처럼 크고 자주 오가는 데이터를 다룰 때 사용하며, 한 번 오갈 때 지연이 짧아야 하는 처리나 여러 프로세스가 같은 데이터를 동시에 읽어야 할 때 사용합니다.

다만 여러 프로세스가 같은 데이터를 동시에 읽기만 하는 게 아니라 누군가가 동시에 쓰게 되면 데이터가 깨질 수가 있기 때문에, 뒤에서 설명할 뮤텍스 · 세마포어와 같은 동기화 수단이 필요합니다.

2-2

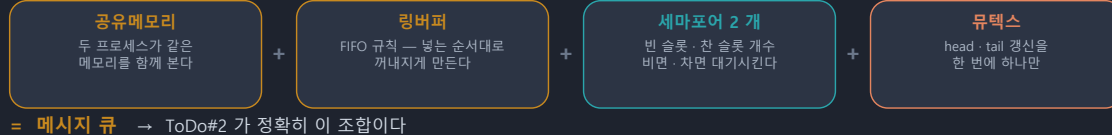
세 가지 비교 — 그리고 조합

메시지 큐는 커널이 주는 기능, 링버퍼는 직접 짜는 자료구조, 공유메모리는 그 링버퍼를 올릴 메모리다

구분	메시지 큐	링버퍼	공유메모리
무엇인가	커널이 관리하는 IPC 객체	자료구조 (IPC 수단 아님)	커널이 연결해 주는 메모리
데이터 단위	메시지 1 개 — 넣은 만큼 나온다	슬롯 1 칸 — 크기 고정	바이트 — 경계는 내가 만든다
복사 횟수	2 회 — 내 변수 → 커널 → 상대	1 회 — 내 변수 → 슬롯	0 회 — 옮겨 담는 일이 없다
동기화	커널이 내장	직접 구현	직접 구현 (필수)
속도	보통	빠름	가장 빠름
용량	커널 메모리라 한도가 있다 (16KB)	고정 — 배열 크기가 곧 용량	물리 메모리가 허용하는 만큼
대표 용도	명령 · 이벤트 전달	들어오는 데이터 임시 보관	대용량 데이터 공유

메시지 큐가 없으면 — 이 넷으로 직접 만든다

세마포어 · 뮤텍스 = 표 동기화 행의 직접 구현 — 다음 장 2-3 에서 설명



발표 대본

2-2 ④ 세 가지 비교

(삭제 — 발표에서 다루지 않음)

2-3

뮤텁스 Mutex (MUTual EXclusion)

상호배제 — 임계구역에 한 번에 하나만 들어가게 하는 자물쇠



발표 대본

2-3 ① 뮤텁스 · 약 1분 50초

이제 동기화 기법 중 하나인 뮤텁스입니다. 뮤텁스는 상호 배제, 영어로 Mutual Exclusion 의 약자입니다.

두 쓰레드가 같은 데이터를 동시에 고치면 값이 깨지게 됩니다. 이렇게 겹치면 깨지는 코드 구간을 임계 구역, Critical Section 이라고 합니다.

뮤텁스는 그 구간에 한 번에 한 명만 들어가게 하는 자물쇠입니다. 들어가기 전에 잠그고, 나오면서 풀니다. 그림처럼 쓰레드 1 이 먼저 잡으면 쓰레드 2 는 풀릴 때까지 기다립니다.

뮤텁스의 특징은 상태가 잠김과 풀림 둘뿐이고, 소유권이 있어서 잠근 쪽만 풀 수 있습니다. 그리고 Windows 에서는 한 프로세스 안이면 CRITICAL_SECTION, 프로세스 사이면 Named Mutex 로 걸립니다. 본 과제에서 ToDo#1 은 CRITICAL_SECTION, ToDo#2 는 Named Mutex 를 사용했습니다.

현재 쓰고 있는 공유 데이터를 다른 쪽에서 건드리지 못하게 하려면 그 부분을 lock 과 unlock 으로 감싸 주면 됩니다. 그리고 아래의 API 는 각 환경별로 뮤텁스를 초기화하고 잠그고 푸는 과정을 설명한 것이니 참고하시면 됩니다.

뮤텁스의 장점은 사용이 단순해서 실수가 적고, 소유권이 있기 때문에 누가 쥐고 있는지 분명해서 문제가 생겼을 때 추적이 된다는 것입니다. 하지만 잠은 쪽만 풀 수 있어서 신호 전달에는 못 쓰고, 락을 잡는 순서가 엇갈리면 데드락이 발생한다는 단점이 있습니다.

2-3 세마포어 Semaphore

쓸 수 있는 자원이 몇 개 남았는지 세는 카운터. wait 로 하나 가져가고 post 로 하나 돌려준다



발표 대본

2-3 ② 세마포어 · 약 1분 20초

세마포어는 쓸 수 있는 자원이 몇 개 남았는지 세는 카운터이고, 동작은 두 가지입니다. wait 로 하나 가져가면서 카운터를 1 줄이고, 0 이면 남을 때까지 기다립니다. post 는 하나 돌려주면서 카운터를 1 늘리고, 기다리던 쪽을 깨웁니다.

뮤텍스와 다른 점은 소유권이 없다는 것입니다. A 가 내리고 B 가 올려도 됩니다. 그래서 다 됐다고 알려 주는 신호로 쓸 수 있습니다. 카운터가 0 이면 커널이 재워 뒀다가 post 때 깨워 줍니다. 그렇기 때문에 기다리는 동안에도 CPU 를 태우는 바쁜 대기, busy wait 이 없습니다.

그래서 세마포어는 커넥션 풀이나 버퍼 슬롯처럼 개수가 정해진 자원, 그리고 한쪽이 채우고 다른 쪽이 꺼낼 때 찻다 비었다를 알리는 곳에서 사용합니다.

장점은 개수를 셀 수 있고, 커널이 깨워 주므로 CPU 낭비가 없고, 소유권이 없어 남이 신호를 줄 수 있고, 이름을 주면 프로세스 사이에서도 쓸 수 있습니다. 단점은 소유자가 없어 잘못 쓴 지점을 찾기 어렵고, post 를 한 번 빠뜨리면 영원히 기다리게 된다는 것입니다.

2-3

뮉텍스 vs 세마포어 — 왜 둘 다 쓰는가

상호배제는 뮉텍스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 반드시 문제가 생긴다

구분	뮉텍스	세마포어
값	잠김 / 풀림 (0 · 1)	0 이상의 정수 카운터
소유권	있다 — 잠근 쪽만 해제	없다 — 누구나 post 가능
목적	상호배제 (데이터 보호)	신호 · 개수 관리 (흐름 제어)
던지는 질문	지금 나만 만지고 있나?	쓸 수 있는 게 몇 개 남았나?
대표 용도	임계구역 보호	송신 - 수신 버퍼, 자원 풀



뮉텍스만 쓰면

버퍼가 빌 때까지 기다리려면 락을 잡았다 풀었다 확인을 반복 → 바쁜 대기로 CPU 를 태운다



세마포어만 쓰면

개수는 맞지만 head · tail 을 두 스레드가 동시에 고쳐 데이터가 깨진다



둘을 같이 쓰면

세마포어로 기다리고, 뮉텍스로 보호한다 → 바쁜 대기도 없고 데이터도 안전하다

송신 - 수신 표준 순서

송신

sem_wait(empty)

>

lock(mutex)

>

버퍼에 write

>

unlock(mutex)

>

sem_post(full)

수신

sem_wait(full)

>

lock(mutex)

>

버퍼에서 read

>

unlock(mutex)

>

sem_post(empty)

순서를 뒤집으면 데드락 — 뮉텍스를 먼저 잡고 세마포어를 기다리면, 상대는 그 락을 영원히 잡지 못한다.

발표 대본

(삭제 — 발표에서 다루지 않음)

2-3 ③ 뮉텍스 vs 세마포어

03

데이터 송 / 수신 기능 구현 및 코드 정리

개발 환경

Windows · Visual Studio 2022 (v143) · x64개발언어 C — IPC 코어는 전부 C DLL (Dynamic Link Library), 확인용 UI 만 MFC

공통 데이터 시나리오 (세 ToDo 모두 동일)

송신측 1로 초기화된 int 를 0.1 초 간격으로 송신, 이후 1씩 증가 (100 까지)수신측 받은 데이터를 그대로 출력

ToDo#1

쓰레드 간 메시지 송 / 수신

전역변수 + 뮤텍스 / 세마포어

`IpcThread.c`

ToDo#2

프로세스 간 메시지 송 / 수신

메시지 큐 (공유메모리 + 링버퍼)

`IpcMsgQ.c`

ToDo#3

다른 프로그램과 메시지 송 / 수신

TCP / IP (SocketUtility 포팅)

`IpcSocket.c`

IPC 로직은 전부 `IpcCore.dll` 안에 있고, MFC (Microsoft Foundation Class) UI (User Interface) 는 버튼으로 코어 함수를 호출하고 로그만 표시한다.

발표 대본

섹션 3 · 약 25초

이제 구현 내용입니다.

ToDo#1에서는 쓰레드 간의 송수신 구현 내용이고, 전역변수와 뮤텍스 · 세마포어를 사용합니다.

ToDo#2에서는 프로세스 간 송수신 구현 내용이고, 공유메모리와 링버퍼를 이용해 메시지 큐를 구현합니다.

ToDo#3에서는 다른 프로그램 간 송수신 구현 내용이고, TCP/IP 소켓 통신을 구현합니다.

ToDo#1

쓰레드 간 송 / 수신 구현

전역 링버퍼를 뮤텝으로 보호하고, 빈 슬롯 · 찬 슬롯 개수를 세마포어 2 개로 센다 · lpcThread.c



LINUX 대응

CRITICAL_SECTION → pthread_mutex_t

CreateSemaphore / WaitForSingleObject → sem_init / sem_wait

ReleaseSemaphore → sem_post

_beginthreadex → pthread_create

※ CreateThread 대신 _beginthreadex — CRT (C Runtime) 자원이 정리되도록

f_IpcThreadQueueSend()

```
// ① 빈 슬롯 대기 - 가득 차면 여기서 블록
if (WaitForSingleObject(s_hSemEmpty, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 뮤텝으로 링버퍼 상호배제
EnterCriticalSection(&s_stRingLock);
s_staRing[s_iRingTail] = *stpMsg;
s_iRingTail = (s_iRingTail + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount++;
LeaveCriticalSection(&s_stRingLock);

// ③ 찬 슬롯 +1 - 수신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemFull, 1, NULL);
```

f_IpcThreadQueueRecv()

```
// ① 찬 슬롯 대기 - 비어 있으면 여기서 블록
if (WaitForSingleObject(s_hSemFull, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 같은 뮤텝으로 상호배제
EnterCriticalSection(&s_stRingLock);
*stpMsg = s_staRing[s_iRingHead];
s_iRingHead = (s_iRingHead + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount--;
LeaveCriticalSection(&s_stRingLock);

// ③ 빈 슬롯 +1 - 송신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemEmpty, 1, NULL);
```

발표 대본

ToDo#1 구현 · 약 1분 20초

먼저 ToDo#1 은 쓰레드 간 송수신입니다. 전역변수에 뮤텝 · 세마포어를 쓰는 것이 과제 조건이었기 때문에, 왼쪽 구조를 보시면 송신 쓰레드와 수신 쓰레드 사이에 전역 링버퍼 8칸을 두었습니다.

그리고 동기화 객체가 세 개입니다. 뮤텝 역할의 CRITICAL_SECTION 하나로 head 와 tail 을 보호하고, 세마포어 두 개로 빈 슬롯과 찬 슬롯의 개수를 셉니다. 빈 슬롯 세마포어는 초기값 8, 찬 슬롯 세마포어는 초기값 0 으로 시작합니다.

오른쪽이 실제 코드인데요. 송신은 세 단계입니다. 먼저 빈 슬롯 세마포어를 기다립니다. 버퍼가 가득 차 있으면 여기서 블록됩니다. 그다음 뮤텝을 잡고 링버퍼에 쓰고 tail 을 하나 증가시킵니다. 마지막으로 찬 슬롯 세마포어를 올려서 수신 쓰레드를 깨웁니다.

수신은 정확히 대칭입니다. 찬 슬롯 기다리고, 같은 뮤텝으로 보호하고, 읽고, 빈 슬롯을 올립니다.

왼쪽 아래는 LINUX 대응입니다. 각 Windows 함수들이 Linux 에서 어떻게 대체되는가, 참고하시면 됩니다.

ToDo#1

실행 결과

한 프로세스 안에서 [Start] — 교대는 0.1 초 간격 때문이고, 송신은 8 건까지 앞서갈 수 있다

```

● IpcUI — 로그창 · Thread [Start] → [Stop]

14:20:31 [TH] start
14:20:31 [TH] tx 1
14:20:31 [TH] rx 1
14:20:31 [TH] tx 2
14:20:31 [TH] rx 2
14:20:31 [TH] tx 3
14:20:31 [TH] rx 3
14:20:32 [TH] tx 4
14:20:32 [TH] rx 4
14:20:32 [TH] tx 5
14:20:32 [TH] rx 5
...
14:20:41 [TH] tx 98
14:20:41 [TH] rx 98
14:20:41 [TH] tx 99
14:20:41 [TH] rx 99
14:20:41 [TH] tx 100
14:20:41 [TH] rx 100
14:20:41 [TH] tx done (100)
14:20:46 [TH] rx done (100, seq err 0)
14:20:46 [TH] stop
  
```

무엇을 확인했나

1 1 부터 100 까지 빠짐없이

송신 100 건, 수신 100 건. 순서가 뒤바뀌거나 빠진 값이 없다.

2 교대는 보장이 아니다

0.1 초 간격이라 큐가 비어서 교대로 보일 뿐이다. 송신은 빈 슬롯 8 개만큼 앞서갈 수 있고 (9 번째부터 대기 — 흐름 제어), 로그 줄 순서는 두 스레드가 각자 찍는 것이라 뒤집힐 수 있다

3 대기 중에도 [Stop] 에 즉시 반응

수신 스레드는 100ms 단위로 타임아웃을 두고 대기하므로 정지 요청이 바로 먹힌다.

실행 방법

IpcUI.exe 를 실행하고 Thread 영역의 [Start] 버튼 하나만 누르면 된다. 한 프로세스 안에서 송신 스레드와 수신 스레드가 한 쌍으로 뜬다.

발표 대본

ToDo#1 실행 결과 · 약 1분

ToDo#1 실행 결과입니다.

MFC 로 UI 를 구현해서 실행해 봤는데요. UI 창에서 Thread 영역의 Start 버튼을 누르면 한 프로세스 안에서 송신 스레드와 수신 스레드가 한 쌍으로 뜬다. 로그를 보시면 start 이후 tx 1, rx 1, tx 2, rx 2 순서로 0.1초 간격으로 찍히고, 100 까지 간 뒤 tx done (100) 이 나옵니다.

지금은 송신이 큐를 채우자마자 수신이 매번 비워 가기 때문에 tx 와 rx 가 정확히 번갈아 가면서 출력되었는데, 뮤텍스와 세마포어는 유실과 중복이 없게 하고 FIFO 순서 보존만을 보장하기 때문에 tx 나 rx 가 연속으로 여러 번 나올 수 있습니다. 그게 버퍼가 있는 이유이고, 이번 과제에서 설정한 8건까지 연속해서 나올 수 있습니다.

ToDo#2

메시지 큐 구현

두 프로세스가 같은 이름으로 커널의 물건 4 개를 찾아 붙는다 — 공유메모리 1 · 뮤텍스 1 · 세마포어 2 · lpcMsgQ.c



LINUX 대응

CreateFileMapping / MapViewOfFile → shmget / shmat
 CreateMutex → pthread_mutex_t
 CreateSemaphore / WaitForSingleObject → sem_open / sem_wait
 CloseHandle → shmctl / shmctl

※ Windows 에는 리눅스의 메시지 큐가 없어서, 위 조합으로 같은 동작을 직접 만들었다

```
f_IpcMsgQSend() = msgsnd()
// @ 빈 슬롯 대기 (프로세스 간 세마포어)
WaitForSingleObject(stpQ->vpSemEmpty, uiTimeout_ms);

// @ 프로세스 간 상호배제 - 네임드 뮤텍스
WaitForSingleObject(stpQ->vpMutex, 50000);

// @ 공유메모리 위의 링버퍼에 write
stpRing->staSlot[stpRing->iTail] = *stpMsg;
stpRing->iTail = (stpRing->iTail + 1) % stpRing->iCapacity;
stpRing->nCount++;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemFull, 1, NULL); // @ 찬 슬롯 +1
```

```
f_IpcMsgQRecv() = msgrcv()
// @ 찬 슬롯 대기
WaitForSingleObject(stpQ->vpSemFull, uiTimeout_ms);

// @ 같은 네임드 뮤텍스
WaitForSingleObject(stpQ->vpMutex, 50000);

// @ 링버퍼에서 read
*stpMsg = stpRing->staSlot[stpRing->iHead];
stpRing->iHead = (stpRing->iHead + 1) % stpRing->iCapacity;
stpRing->nCount--;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemEmpty, 1, NULL); // @ 빈 슬롯 +1
```

발표 대본

ToDo#2 구현 · 약 1분 15초

ToDo#2 는 프로세스 간 송수신입니다. 앞서 말씀드린 것처럼 Windows 에는 Linux 의 System V 메시지 큐 기능이 없기 때문에, 공유메모리와 링버퍼에 뮤텍스 · 세마포어를 얹어 만들었습니다.

왼쪽 구조를 보시면 ToDo#1 과 모양이 같지만, 이번엔 쓰레드가 아니라 프로세스라는 점이 다릅니다.

아까는 쓰레드끼리라서 전역변수에 링버퍼를 뒀지만, 이번에는 프로세스 간 통신이라 링버퍼를 두 프로세스의 바깥인 OS 가 내주는 공유메모리에 올립니다. 그러면 그 프로세스가 그 공유메모리를 찾을 때 이름으로 찾을 수 있습니다. 양쪽이 lpcDemoQ 라는 같은 이름을 대면 OS 가 같은 물건을 연결해 줍니다.

그래서 공유메모리, 뮤텍스, 세마포어를 각각의 이름으로 찾아야 합니다. 이름을 구별해야 하기 때문에 앞에 lpcDemoQ 는 똑같이 두고 뒤에만 .map, .mtx, .sem.e, .sem.f 와 같은 꼬리표를 달았습니다.

그 외에는 ToDo#1 과 동일합니다. 오른쪽 실제 코드 동작 단계도 동일합니다.

ToDo#2

실행 결과

[New Process] 로 창을 하나 더 띄우고, 한쪽은 [Recv] · 다른쪽은 [Send] 를 누른다

프로세스 ① 수신 — [Recv] 를 먼저 눌러 큐를 만든다

```

14:22:05 [MQ] queue 'IpcDemoQ' open
14:22:05 [MQ] start (rx)
14:22:12 [MQ] rx 1
14:22:12 [MQ] rx 2
14:22:12 [MQ] rx 3
14:22:12 [MQ] rx 4
14:22:12 [MQ] rx 5
14:22:13 [MQ] rx 6
...
14:22:22 [MQ] rx 98
14:22:22 [MQ] rx 99
14:22:22 [MQ] rx 100
14:22:28 [MQ] rx done (100)
14:22:28 [MQ] stop

```

프로세스 ② 송신 — [New Process] 로 띄운 창에서 [Send]

```

14:22:12 [MQ] queue 'IpcDemoQ' open
14:22:12 [MQ] start (tx)
14:22:12 [MQ] tx 1
14:22:12 [MQ] tx 2
14:22:12 [MQ] tx 3
14:22:12 [MQ] tx 4
14:22:12 [MQ] tx 5
14:22:13 [MQ] tx 6
...
14:22:22 [MQ] tx 98
14:22:22 [MQ] tx 99
14:22:22 [MQ] tx 100
14:22:22 [MQ] tx done (100)
14:22:26 [MQ] stop

```

1

두 프로세스 사이로 100 건이 그대로 전달

송신 창의 tx 1~100 과 수신 창의 rx 1~100 이 정확히 일치한다.

2

수신을 먼저 눌러도, 송신을 먼저 눌러도 동작

Create 가 없으면 만들고 있으면 참여하므로 기동 순서에 의존하지 않는다.

3

수신 창의 시각이 7 초 늦게 시작

큐가 먼저 만들어져 대기하고 있다가, 송신이 붙는 순간부터 받기 시작한다 — 비동기 동작 확인.

발표 대본

ToDo#2 실행 결과 · 약 1분 10초

ToDo#2 실행 결과입니다.

UI 에서 New Process 버튼으로 같은 프로그램 창을 하나 더 띄우고, 한쪽에서 Recv, 다른쪽에서 Send 버튼을 누르면 됩니다.

왼쪽이 수신 프로세스인데, 22분 05초에 queue IpcDemoQ open 이 찍히고 rx 로 시작합니다. 오른쪽 송신 프로세스는 7초 뒤인 22분 12초에 같은 큐에 붙어서 tx 1 부터 보내기 시작합니다.

여기서 확인할 수 있는 것은, 먼저 송신 창의 tx 1 부터 100 까지와 수신 창의 rx 1 부터 100 까지가 정확히 일치합니다. 두 프로세스 사이에 100 건이 그대로 전달됩니다.

그리고 수신을 먼저 눌러도 송신을 먼저 눌러도 동작합니다. Create 가 없으면 만들고 있으면 참여하기 때문에 누가 먼저 동작했느냐는 상관없습니다.

마지막으로 수신 쪽이 7초 먼저 떴서 큐를 만들어 놓고 대기하다가 송신이 붙는 순간부터 받기 시작하는데, 이것이 메시지 큐의 비동기 성질입니다.

ToDo#3

TCP / IP 구현

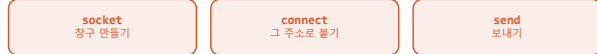
다른 PC · 다른 프로그램과의 송수신 — Linux SocketUtility v5.0 → Winsock2 포팅, 함수명 · 인자 · 상수는 원본 그대로 · lpcSocket.c

연결 맺는 순서 — 문을 열고 기다리는 쪽이 서버, 찾아가서 두드리는 쪽이 클라이언트

① 먼저 Rx · 수신 = 서버 [Recv] — 자리를 잡고 기다린다



② 그다음 Tx · 송신 = 클라이언트 [Send] — 그 자리를 찾아간다



※ accept 는 200ms 단위 select 루프, connect 는 논블로킹 + select — [Stop] 에 바로 반응한다

리눅스 원본을 Windows 로 옮기며 걸린 것 — 컴파일은 통과하는데 뜻이 달라지는 항목

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) · x64 에서 64bit
송 / 수신 타임아웃	timeval 구조체	DWORD 밀리초 — 그대로 넘기면 엉뚱한 값
소켓 닫기	close()	closesocket()
초기화	없음	WSAStartup / WSACleanup 필요
에러 확인	errno	WSAGetLastError()
select 1 번 인자	maxfd + 1	무시된다 (있어도 되고 없어도 된다)

※ 타임아웃은 형이 맞아 보여 그냥 지나치기 쉽다 — 내부에서 timeval → DWORD ms 로 변환했다

f_SocketRecvTCP_IPv4() — TCP 는 바이트 스트림, 다 채울 때까지 반복

```
send(4 byte) 를 한 번 불러도 recv 는 2 byte 만 받을 수 있다 — 메시지 경계가 없다
// ① 요청한 크기를 다 채울 때까지 반복해서 받는다
while ((lRemain > 0) && (stpSocket->iSockStatus == Connected)) {
    iRecv = recv((SOCKET)stpSocket->hSockId, cpCursor, iChunk, 0);

    if (iRecv > 0) {
        cpCursor += iRecv;
        lTotalRecv += iRecv;
        lRemain = lFixedDataSize - lTotalRecv;
    }
    else stpSocket->iSockStatus = Disconnected; // ② 상태 종료 · 에러
}
```

f_TcpSenderProc() — 데모 송신, 프레임 헤더 없이 int 4 byte

```
// 1 부터 100 까지 0.1 초 간격으로 보낸다
for (iData = IPC_DEMO_FIRST_VALUE;
    iData <= IPC_DEMO_LAST_VALUE; iData++) {
    // htonl 체크박스가 켜져 있으면 네트워크 바이트 순서로 싣는다
    iWire = (s_iDemoUseNBO != 0) ? htonl(iData) : iData;
    lSent = f_SocketSendTCP_IPv4(&s_stDemoSocket, &iWire, sizeof(iWire));
    // 0.1 초 쉬면서 정지 요청도 같이 확인한다
    f_IsDemoStopRequested(IPC_DEMO_INTERVAL_MS);
}
```

경계는 없애 정한다 — ① 길이 붙이기 ② 구분자 ③ 고정 길이 중 본 과정은 ③ 번 양쪽 다 little-endian 이라 그대로 말고, 상대가 htonl 규약이면 체크박스를 켜다 보내는 쪽 f_SocketSendTCP_IPv4() 도 같은 모양 — 다 보낼 때까지 반복한다

발표 대본

ToDo#3 구현 · 약 1분 15초

ToDo#3 은 다른 PC, 다른 프로그램과 송수신하는 것이고 TCP/IP 로 구현했습니다. 첨부받은 Linux SocketUtility 를 윈도우의 Winsock2 라이브러리로 포팅했습니다.

좌측 상단에 연결 맺는 순서를 보시면, 소켓은 전화와 비슷해서 한쪽이 번호를 열어 두고 기다려야 다른 쪽이 걸 수 있습니다. 기다리는 쪽이 서버, 거는 쪽이 클라이언트입니다. 그래서 수신 Rx 가 서버로 socket, bind, listen, accept 로 먼저 자리를 잡고, 송신 Tx 가 클라이언트로 connect 해서 찾아갑니다. 그래서 UI 에서 Recv 를 먼저 눌러야 합니다. Send 를 먼저 하면 connect 를 실패합니다.

오른쪽의 함수 코드들을 보시면, TCP 는 바이트 스트림이라 메시지 경계가 없어서 4바이트를 한 번에 보내도 수신은 2바이트만 받을 수 있다고 설명했었습니다. 그래서 다 채울 때까지 반복해서 받아야 하고, f_SocketRecvTCP_IPv4 의 while 루프가 그 일을 합니다.

ToDo#3

실행 결과

한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트) — 다른 PC 의 프로그램과도 같은 방식으로 붙는다

프로세스 ① 수신 = 서버 — [Recv]

```

14:25:02 [TCP] start (rx)
14:25:02 [TCP] listen 0.0.0.0:51000
14:25:09 [TCP] accept 127.0.0.1:60312
14:25:09 [TCP] rx 1
14:25:09 [TCP] rx 2
14:25:09 [TCP] rx 3
14:25:10 [TCP] rx 4
14:25:10 [TCP] rx 5
...
14:25:19 [TCP] rx 98
14:25:19 [TCP] rx 99
14:25:19 [TCP] rx 100
14:25:20 [TCP] peer closed
14:25:20 [TCP] rx done (100)

```

프로세스 ② 송신 = 클라이언트 — [Send]

```

14:25:09 [TCP] start (tx)
14:25:09 [TCP] connecting 127.0.0.1:51000
14:25:09 [TCP] connected 127.0.0.1:51000
14:25:09 [TCP] tx 1
14:25:09 [TCP] tx 2
14:25:09 [TCP] tx 3
14:25:10 [TCP] tx 4
14:25:10 [TCP] tx 5
...
14:25:19 [TCP] tx 98
14:25:19 [TCP] tx 99
14:25:19 [TCP] tx 100
14:25:19 [TCP] tx done (100)
14:25:20 [TCP] stop

```

1 연결 → 100 건 → 정상 종료

accept 로 상대 IP · 포트를 확인하고, 100 건을 받은 뒤 상대가 끊으면 peer closed 로 마무리된다.

2 다른 PC 와 붙일 때

수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 연다. 상대가 htonl 을 쓰면 체크박스를 켜다.

3 역할이 반대인 상대와도 가능

Tx / Rx Init 함수를 바꿔 부르지만 하면 된다. 함수명 · 상수는 원본 SocketUtility 와 동일하게 두었다.

발표 대본

ToDo#3 실행 결과 · 약 45초

ToDo#3 실행 결과입니다.

마찬가지로 창을 두 개 띄우고 한쪽은 Recv 로 서버, 다른쪽은 Send 로 클라이언트를 시작합니다. 그리고 htonl 은 빅 엔디안이나 리틀 엔디안이나 인데, 양쪽의 설정을 동일하게 맞춰 줘야 합니다.

왼쪽 로그를 보시면 listen 0.0.0.0:51000 으로 대기하다가, 7초 뒤 클라이언트가 붙으면서 accept 에 상대 IP 와 포트가 찍힙니다. 그리고 rx 1 부터 100 까지 받습니다.

클라이언트가 100 건을 다 보내고 소켓을 닫으면 peer closed 가 뜨고 rx done (100) 으로 마무리됩니다.

정리

개념 세 가지와, 그 개념이 코드에서 어떻게 만났는지

1

뮤텍스와 세마포어는 짝이다

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어.
하나로 다른 하나를 흉내내면 바쁜 대기나 데이터가 깨진다.

2

메시지 큐는 없으면 만들 수 있다

공유메모리 + 링버퍼(FIFO) + 세마포어 2 개 + 뮤텍스.
Windows 에 System V 메시지 큐가 없어 이 조합으로 msgsnd / msgrcv 와 같은 의미를 만들었다.

3

포팅에서 위험한 것은 타입이 맞아떨어지는 차이다

SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을
먼저 찾아야 한다. 소켓 핸들 크기, 타임아웃 단위가 대표적이다.

검증 결과 ToDo#1 · #2 · #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고, 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했다.

발표 대본

정리

(대본 미작성)

발표에서는 말하지 않는다. 질의응답에서 표의 각 행을 파고드는 질문이 나올 때 쓴다.

대본합본 PDF 각 쪽 하단의 「예상 질문」은 이 내용을 한 줄로 줄인 것이고, 아래가 그 자세한 버전이다.

각 항목은 짧은 답(그대로 말할 수 있는 한두 문장)과 더 물어보면(배경 · 숫자 · 코드 연결점)으로 나뉘 두었다.